

26. Съхранение и обмен на данни при мобилни приложения. Синхронен и асинхронен достъп. Употреба на класовете `ContentProvider` и `ContentResolver` в Android.

1. Увод

Мобилните приложения работят в среда с ограничени ресурси (памет, батерия, процесорна мощ) и с изисквания за отзивчив потребителски интерфейс. Затова платформата Android предлага няколко механизма за **съхранение на данни** (locally на устройството или отдалечено) и стриктен модел за **достъп до данни**, който разграничава синхронни и асинхронни операции, за да не се блокира главната (UI) нишка. Когато данните трябва да бъдат достъпни от няколко приложения, Android предоставя специализиран компонент - **`ContentProvider`**, заедно с клиентския му интерфейс **`ContentResolver`**.

2. Начини за съхранение

Механизъм	Предназначение	Характеристики
<code>SharedPreferences</code>	Малки по обем ключ-стойност данни (настройки, флагове)	Съхранява се в XML файл, private за приложението
<code>Internal Storage</code>	Файлове, специфични за приложението	Директория <code>/data/data/<package>/files</code> , изтрива се при деинсталация

Механизъм	Предназначение	Характеристики
External Storage	По-големи файлове (медия, документи)	Изисква runtime permissions от Android 6.0 нататък
SQLite / Room	Структурирани, релационни данни	Room е ORM слой над SQLite, препоръчан от Google (Jetpack)
Мрежово съхранение	Данни, споделени между устройства	REST/GraphQL API, Firebase, синхронизация чрез WorkManager
ContentProvider	Споделяне на данни между приложения	Абстрахира конкретния storage слой зад унифициран API

Изборът на механизъм зависи от обема, структурата, времето на живот и необходимостта от споделяне на данните с други приложения.

3. Синхронен и асинхронен достъп до данни

3.1 Проблемът с главната нишка

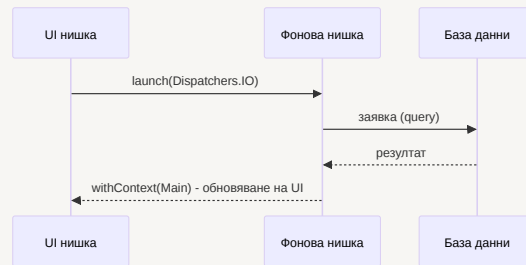
Android изпълнява UI операциите в една **единствена главна (UI) нишка**. Всяка операция, която отнема повече от ~5 секунди в тази нишка (I/O, мрежови повиквания, тежки изчисления), води до ANR (Application Not Responding). Затова:

- **Синхронен достъп** (блокиращ) е допустим само за бързи, локални операции или в извън-UI нишка.
- **Асинхронен достъп** (неблокиращ) е задължителен за I/O, мрежа и достъп до бази данни, така че UI нишката да остане свободна.

3.2 Механизми за асинхронност в Android

- **Thread / Runnable** - Класически Java подход, изисква ръчна синхронизация с UI чрез `Handler`.
- **AsyncTask** - Специализиран клас в Android за лесно изпълнение на фонові задачи и публикуване на резултатите в UI нишката.

- **Executor / ExecutorService** - управление на пул от нишки, комбинирано с **Handler** за връщане на резултат в UI нишката.



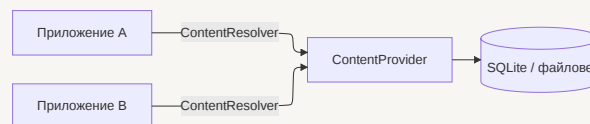
4. ContentProvider и ContentResolver

4.1 Предназначение

ContentProvider е компонент от четирите основни градивни блока на Android (заедно с Activity, Service, BroadcastReceiver), чиято задача е да **абстрахира достъпа до структурирани данни** и да позволи **споделянето им между различни приложения**, независимо от вътрешния начин на съхранение (SQLite, файлове, мрежа). Класически системни примери са **ContactsContract** (контакти) и **MediaStore** (медийни файлове).

ContentResolver е клиентският клас, чрез който всяко приложение (включително самото притежаващо данните) осъществява достъп до даден **ContentProvider**, без да познава детайлите по имплементацията му.

4.2 Архитектура



Комуникацията минава винаги през ContentResolver → ContentProvider, което позволява на Android да контролира правата за достъп (permissions) централизирано, независимо кое приложение е собственик на данните.

4.3 URI и идентификация на данни

Всеки ContentProvider се идентифицира чрез уникален authority, а конкретните ресурси — чрез URI от вида:

```
content://<authority>/<path>/<id>
```

Например: `content://com.android.contacts/contacts/45`

- **Схема (`content://`)**: Стандартен префикс, който указва на Android, че този URI адресира `ContentProvider`.
- **Authority (Орган/Идентификатор)**: Уникален низ (обикновено обратно домейн име на пакета), който идентифицира конкретния доставчик в системата.
- **Path (Път)**: Идентифицира типа данни или таблицата (например `products`).
- **ID (Идентификатор на ред)**: Незадължителен параметър за насочване към конкретен запис (например `/5`).

4.4 Основни методи на ContentProvider

При създаване на собствен ContentProvider се наследява класът ContentProvider и се имплементират:

```
public class TaskProvider extends ContentProvider {  
  
    private static final int TASKS = 1;  
    private static final int TASK_ID = 2;  
    private static final String AUTHORITY = "com.example.app.  
provider";  
    private static final UriMatcher uriMatcher = new UriMatch  
er(UriMatcher.NO_MATCH);  
  
    static {  
        uriMatcher.addURI(AUTHORITY, "tasks", TASKS);  
        uriMatcher.addURI(AUTHORITY, "tasks/#", TASK_ID);  
    }  
}
```

```

@Override
public boolean onCreate() {
    // инициализация на връзка с базата данни
    return true;
}

@Override
public Cursor query(Uri uri, String[] projection, String
selection,
                    String[] selectionArgs, String sortOrder) {
    switch (uriMatcher.match(uri)) {
        case TASKS:
            return db.query("tasks", projection, selection,
selectionArgs, null, null, sortOrder);
        case TASK_ID:
            String id = uri.getLastPathSegment();
            return db.query("tasks", projection, "_id=?",
new String[]{id}, null, null, null);
        default:
            throw new IllegalArgumentException("Непознат
URI: " + uri);
    }
}

@Override
public Uri insert(Uri uri, ContentValues values) {
    long id = db.insert("tasks", null, values);
    Uri newUri = ContentUris.withAppendedId(uri, id);
    getContext().getContentResolver().notifyChange(newUri,
null); // уведомяване за промяна
    return newUri;
}

@Override

```

```

    public int update(Uri uri, ContentValues values, String selection, String[] selectionArgs) { /* ... */ return 0; }

    @Override
    public int delete(Uri uri, String selection, String[] selectionArgs) { /* ... */ return 0; }

    @Override
    public String getType(Uri uri) {
        return "vnd.android.cursor.dir/vnd.example.tasks";
    }
}

```

Като Provider-ът трябва да се регистрира в `AndroidManifest.xml` :

```

<provider
    android:name=".TaskProvider"
    android:authorities="com.example.app.provider"
    android:exported="true"
    android:readPermission="com.example.app.permission.READ_TASKS"
    android:writePermission="com.example.app.permission.WRITE_TASKS" />

```

Извличането на данните през ContentResolver става чрез:

```

val uri = Uri.parse("content://com.example.app.provider/tasks")

// Синхронна заявка (трябва да се изпълни във фонова нишка!)
val cursor = contentResolver.query(uri, null, null, null, null)
cursor?.use {
    while (it.moveToNext()) {
        val title = it.getString(it.getColumnIndexOrThrow("title"))
    }
}

```

```

    }
}

// Вмъкване на нов запис
val values = ContentValues().apply {
    put("title", "Нова задача")
}
contentResolver.insert(uri, values)

```

Тъй като `query()`, `insert()`, `update()` и `delete()` на `ContentResolver` са потенциално блокиращи (I/O операции), те трябва да се изпълняват асинхронно - например чрез `CursorLoader` (по-стар подход) или в `Dispatchers.IO` при използване на Coroutines, а промените могат да се наблюдават реактивно чрез `ContentObserver`, регистриран с `registerContentObserver()`.

4.6 Предимства на модела ContentProvider/ContentResolver

- Унифициран, независим от storage слоя API (SQLite, файлове, мрежа).
- Централизиран контрол на достъпа чрез Android permission модела.
- Възможност за наблюдение на промени (`ContentObserver`) и вграждане в `CursorAdapter` / `Loader` архитектурата.
- Стандартен начин за интеграция с системни компоненти (напр. `SearchSuggestionProvider`, синхронизация чрез `SyncAdapter`).